Firmware Supplement

Undocumented Commands, Service Functions and Internals

Community Reverse-Engineering Supplement
Document EPM-90S01
Jared Cabot • jetstreamtechnology@protonmail.com
Firmware analysis and text prepared with Claude (Anthropic)
Licensed MIT
Revision 1.0 • October 2026

Scope

This supplement documents behaviour of the E4418B and E4419B firmware that the Agilent manuals do not describe: the undocumented SCPI commands (most of them under the `DIAGnostic` and `SERVice` subsystems), the HP 436A emulation mode, the measurement-board bus protocol and its ADSP-2101 program, the sensor EEPROM data format, the boot ROM, the front-panel service menus, and the device-specific error codes.

It does not restate the documented command set, the measurement theory or the operating procedures. For those, see the E4418B/E4419B Programming Guide, User's Guide and Service Guide. Where this document touches a documented command, it does so only to record an undocumented option or side effect.

Provenance

The findings come from three sources, named throughout so the reader can tell them apart. Disassembly and decompilation of the main-processor firmware images (Ghidra and a 68k listing) and of the measurement-board DSP image give the command handlers, tables and addresses. Measurement on a real E4419B (serial MY45103686, firmware A2.09.01) over GPIB confirms the behaviour that could be exercised safely; Chapter 11 lists what was run, and everything else comes from the code. The CLIP and Service Guide give the hardware.

Applicability

The addresses and tables are taken from the E4418B image unless a line says E4419B. Both images were built on 12 May 2009. The bench unit runs E4419B firmware A2.09.01 with boot ROM A.02.00. Other firmware revisions will differ in their addresses; the command codes and the structure are expected to hold across the EPM family, but only this revision was checked. The E4418B is single-channel and the E4419B dual-channel; channel-suffix behaviour differs accordingly.

Conventions

Memory addresses and register contents are hexadecimal with a $ prefix: $E05000. SCPI command codes (the firmware's internal command numbers) are also written $-prefixed. The instrument's numeric parser accepts decimal or 0x-prefixed hexadecimal, so command examples appear as they were sent to the instrument. Command text is in a monospaced face; angle brackets mark a parameter, square brackets an optional keyword, as in the Programming Guide. References to the Agilent manuals name the document and section.

Preparation and disclaimer

The firmware analysis, the bench measurements and the text were prepared by Jared Cabot with Claude (Anthropic) and are original to this document. Hardware details are drawn from the Agilent CLIP and Service Guide, cited where used. This document supplements the Agilent E4418B/E4419B manuals and does not replace them.

This is not an Agilent, Hewlett-Packard or Keysight document and carries no endorsement from them. Several commands it describes write the sensor EEPROM, clear battery-backed memory, start a firmware download or change calibration state. Used without care they can leave the instrument or a sensor needing recalibration or a firmware reload. Read the admonitions before sending any command that writes.

Contents

1. General Information **1-1**
About This Supplement 1-1
Instrument Architecture 1-1
Firmware Image Structure 1-2
Processor Memory Map 1-2

2. Remote Operation Extensions **2-1**
The Command-Code Model 2-1
SYSTem:LANGuage Options 2-1
The READD Alias 2-1
The Dormant *_TRG? Query 2-1
HP 436A Emulation 2-2

3. The DIAGnostic Subsystem **3-1**
Overview 3-1
Memory Access 3-1
Battery-Backed Memory 3-2
Firmware Download Entry 3-2
Debug Monitor and RTOS Reports 3-2
Version Queries 3-2
Sensor EEPROM Access 3-3
Sensor Calibration Image 3-3
Measurement Board and DSP 3-4
Measurement Parameter Overrides 3-4
Status Reports 3-5
Display 3-5
Keyboard, Battery and Self Tests 3-6
Interface and Hardware Selection 3-6

4. Other Undocumented Commands **4-1**
SERVice Subsystem 4-1
STATus:DEBug Subsystem 4-1
SENSe:CORRection:STATe 4-1
SENSe:CORRection:LOSS1 4-2
MEMory:TABLe:LOSS 4-2
Stub Commands 4-2
Version-String Side Effects 4-3

Contents (cont'd)

5. The Measurement Board and DSP **5-1**
Bus Protocol 5-1
Channel Configuration Bits 5-2
The ADSP-2101 Program 5-2
The Command Mailbox 5-2
The DSP Dispatch Table 5-3
The XC3042 Gate Array 5-3

6. Sensor EEPROM Data Format **6-1**
Container Layout 6-1
Block-Type Registry 6-1
Severity and Read Failures 6-2
The N8482A EEPROM 6-2

7. Firmware and Memory **7-1**
The Battery-Backed Header 7-1
The Internal Variable Store 7-1
SCPI Parser Structures 7-1
Command-Set Coverage 7-2

8. The Boot ROM **8-1**
Identification 8-1
Power-Up Modes 8-1
The ROM Monitor 8-2
Download Record Format 8-2

9. Front-Panel Service Functions **9-1**
The Softkey Menu Engine 9-1
Reaching the Service Items 9-1
Edit Serial Number 9-1
The Individual Self Tests 9-2
The HP436A Modes Menu 9-2
The Test1 to Test6 Screen 9-2

10. Device Error Codes **10-1**
The Two Translation Tables 10-1
Positive Device Codes 10-1

11. Verification Record **11-1**
Confirmed by Observation 11-1
Not Verified 11-2
How to Reproduce 11-2

1

General Information

About This Supplement

The EPM Programming Guide lists the SCPI commands an application is meant to use. The firmware carries more than that list: service and diagnostic commands used in manufacture and repair, a legacy instrument-emulation mode, and a set of internal tables and structures that the documented commands sit on top of. This supplement records what those extra commands do and how the pieces fit together, at the level of detail a service or development engineer needs.

About 150 of the command codes reachable from the keyword tree appear in none of the Agilent manuals. Most are in the `DIAGnostic` subtree. Chapter 3 covers that subtree; Chapter 4 covers the undocumented commands elsewhere. Chapters 5 through 9 document the hardware and firmware those commands reach: the measurement board and its DSP, the sensor EEPROM format, the memory layout, the boot ROM and the front-panel service menus. Chapter 10 gives the device-specific error codes, and Chapter 11 records what was confirmed on a real instrument.

Caution



The commands in this supplement include ones that write the sensor EEPROM, clear battery-backed memory, start a firmware download and change calibration state. None are needed for normal measurement. Read the entry and its admonitions before sending any command that writes.

Instrument Architecture

The E4418B and E4419B share one processor-board design and differ mainly in the number of measurement channels. Part numbers below are from the CLIP.

- **Main processor:** a Motorola MC68332 (CPU32 core, 16-bit bus) running the pSOS+ real-time kernel. It holds the firmware, the SCPI parser, the display and the front panel. The build path in the image names the project codename `merlin`.
- **Measurement board (A6, part E4418-60003):** one per channel, carrying an Analog Devices ADSP-2101 DSP (flash E4418-80023), a Xilinx XC3042A gate array (part 1821-1672) and the sensor interface. The main processor talks to it over a serial bus; see Chapter 5.
- **Display:** a graphic LCD on an SED1330 controller, 320 pixels wide.
- **Firmware store:** two 4 Mbit flash chips, one per byte lane (E4418-80019 and -80020). The boot ROM occupies the bottom of both chips and the main firmware the rest.

At power-up the MC68332 runs the boot ROM at address $000000. The boot ROM sets up the chip selects and the exception vectors, decides whether to run the main firmware or wait for a download, and jumps to the main firmware at $020000. The main firmware never reprograms the chip selects or the vector base, so it depends on the boot ROM's setup throughout. Chapter 8 covers the boot ROM.

Firmware Image Structure

A firmware download file (`.dload`) is a sequence of load records, each a 10-byte header (`WL`, a 32-bit load address and a 32-bit inclusive end address) followed by the data, ending with a two-byte trailer. Both the E4418B and E4419B files hold two load records: the code at $020000 and the initialised data segment immediately after it. The trailer `48 30` is the boot loader's `H0` end-of-download record (Chapter 8).

Table 1-1. Load records in the download images

Model	Code record	Data record (ROM)	Data copied to RAM
E4418B	$020000	$0E9170 to $0F462B	$E05000 to $E104BB
E4419B	$020000	$0E9BC4 to $0F1D5B	$E05000 to $E0D197

The initialised data segment is copied from ROM to RAM at boot; a RAM address A has its initial value at ROM (A minus $E05000 plus the data record's ROM start). BSS follows the copied data. The data record begins with an image of the battery-backed header. When the first long of that image is $12345678, as in both files, the boot copy starts after the header, so the live header and the settings in it survive a firmware reload (Chapter 7).

Note



The image checks itself. A header at $020030, written to both flash chips, holds the device count, the image length in words and a 16-bit sum of each byte lane at $020054. The ROM Checksum self test recomputes the lane sums over the code and data records. Both sums match the image assembled from the two load records.

Processor Memory Map

The map below is the address space as the main firmware uses it. The boot ROM sets up the MC68332 chip selects; the firmware does not touch them.

Table 1-2. Main-processor address map

Range	Use
$000000 to $01FFFF	Boot ROM A.02.00 (Chapter 8). VBR is set to $00000A.
$020000 to ~$0F4640	Main firmware (the download image)
$E00000 to $E3FFFF	Battery-backed RAM: header, data segment, BSS
$E40000 to $E7FFFF	Further RAM: pSOS regions, heap, stack (top $E7FFF0)
$F00000 to $F00007	GPIB controller, 8 registers
$F00800 to $F00807	16550-type UART for the RS-232/RS-422 option
$F01180, $F011C0	PLD mode register (mode 0 and mode 1)
$F01800, $F01802	Recorder-output DACs
$FFFA11 to $FFFA17	SIM port E/F data and direction
$FFFC15 to $FFFC1F	QSM port and QSPI registers
$FFFE00 to $FFFE20, $FFFF00	TPU registers and parameter RAM

The battery-backed range is inferred from a boot-time check that prints `RUN OUT OF BATTERY-BACKED RAM` if BSS would end above $E40000. The UART address comes from the code; on the bench `DIAG:UART:REGS?` returns a 16550 register file from it.

2

Remote Operation Extensions

The Command-Code Model

The SCPI parser maps every accepted keyword path to a 16-bit command code, and the code to a descriptor that names the set handler and the query handler. The command codes are stable across the keyword tree and are used throughout this supplement to identify a command independently of its spelling. A handler returns a small internal status code; the dispatcher translates that to a SCPI error through one of two tables (Chapter 10). Chapter 7 describes the parser structures in detail.

Two remote behaviours below are undocumented, one is a model-specific alias, and one is a query that cannot be reached as typed.

SYSTem:LANGuage Options

The manuals list `SCPI` and one legacy language per model. The parser accepts more:

Table 2-1. SYSTem:LANGuage values accepted

Model	Accepted values	Documented
E4418B	SCPI, HP436A, HP437, HP437B	SCPI, HP437B (example: HP437)
E4419B	SCPI, HP438, HP438A	SCPI, HP438A

`HP437` (used in the Programming Guide's own example) and `HP438` are short aliases of the long forms. `HP436A` on the E4418B selects a distinct HP 436A emulation with its own parser and front-panel menu, described below.

The READD Alias

The E4419B accepts `READD` as an alias of the `READ` root keyword, with the same command codes. The E4418B tree does not carry it. On the bench, `READD?` behaves as `READ?` (both return `-213 Init ignored` with INIT:CONT ON), where an unknown header would give -113. Not described in the Programming Guide.

The Dormant *_TRG? Query

Command code $086 is a common query that runs the `*TRG` trigger routine and then writes the resulting reading into the response, in effect a triggered read in one query. It cannot be reached from the bus. The keyword entry is literally `_TRG`, with a leading underscore, which is not a legal SCPI program-header character, so sending `*_TRG?` returns `-101 Invalid character` (confirmed on the E4419B). Use `*TRG` followed by a read.

HP 436A Emulation

Selecting `SYSTem:LANGuage HP436A` on the E4418B, or the Command Set front-panel item, puts the instrument into an HP 436A emulation the manuals do not mention. A separate HP-IB driver takes over the GPIB controller, and each received character runs through a small state machine that translates the 436A codes into SCPI commands the normal parser then executes, so each code has the same effect as the SCPI commands listed.

Table 2-2. HP 436A command codes and their SCPI equivalents

Code	436A meaning	SCPI issued
+	Cal factor off	SENSE:CORR:GAIN1 100
-	Cal factor on	SENSE:CORR:GAIN1 <n>
1 to 5	Manual range 1 to 5	range kept in emulation state
9	Autorange	range state set to auto
A	Watts	CALC:REL:STAT OFF, UNIT:POW W
B	dB relative	UNIT:POW DBM, CALC:REL:STAT ON
C	dB reference	UNIT:POW DBM, CALC:REL:MAGN:AUTO ONCE
D	dBm	CALC:REL:STAT OFF if needed, UNIT:POW DBM
H	Hold	TRIG:SOURCE HOLD
I	Trigger immediate	SENSE:AVER:STAT OFF, TRIG:SOUR BUS, TRIG:DEL:AUTO OFF, INIT:CONT ON, TRIG:IMM
R	Free run	SENSE:AVER:STAT ON, TRIG:SOUR IMM, TRIG:DEL:AUTO OFF, INIT:CONT ON
T	Trigger with delay	SENSE:AVER:STAT ON, INIT:CONT OFF, TRIG:SOUR IMM, TRIG:DEL:AUTO ON, INIT:IMM
V	Free run with delay	same as R
Z	Zero	UNIT:POW W, CAL:ZERO:AUTO ONCE

The emulation also accepts codes the original 436A did not have: `FR <value> KZ|MZ|GZ` sets the measurement frequency (for a sensor that uses frequency correction), `X1` and `X2` select the two measurement speeds, and the `SYST:LANG` forms are recognised by a character matcher so the mode can be left without the front panel. The reading output uses the 436A status, range and mode characters. A front-panel **HP436A Modes** menu (Zero Mode, Filter Mode) sets the emulation state; see Chapter 9.

From firmware analysis; the decoder is at $084346. Not described in the Programming Guide.

3

The DIAGnostic Subsystem

Overview

The keyword is `DIAGnostic[1|2]`; the suffix selects the channel and defaults to 1. On the E4418B use 1 or no suffix. The commands below exist in both models unless a line says otherwise. Many handlers also print a line on the debug console as well as returning the SCPI response. Parameters follow the usual converter types: `<num>` accepts MIN/MAX/DEF, `<bool>` is ON|OFF|1|0, and `<char>` is one of a fixed word list.

Command codes and handler addresses are from the E4418B image. None of the commands in this chapter is described in the Programming Guide. Those checked on the E4419B are listed in Chapter 11.

Memory Access

These read and write the MC68332 address space directly, using the physical addresses in the memory map (Table 1-2). A bus error from an unmapped address is not trapped by the handlers.

Table 3-1. Memory-access commands

Command	Code	Behaviour
`DIAGnostic:POKE <addr>,<word>`	$12A	Writes a 16-bit word. Odd address: -224.
`DIAGnostic:PEEK? <addr>`	$12B	Reads a 16-bit word. Odd address: -224.
`DIAGnostic:PEEK:BLOCK? <addr>,<n>`	$12C	Reads n words (n ≤ 256, else -222); 2n bytes, high byte first. Address 0 is rejected with -222; read that word with PEEK?.
`DIAGnostic:PEEK:BYTE? <addr>`	$12D	Reads one byte, any address.
`DIAGnostic:POKE:BYTE <addr>,<byte>`	$12E	Writes one byte, any address.

Confirmed on the E4419B: `PEEK?` and `PEEK:BLOCK?` return the GPIB address word, the power-up magic and the error-table entries, all matching the firmware image, and read the whole 1 MB of flash (Chapter 11).

Caution



`POKE` and `POKE:BYTE` write anywhere in the address space, including the battery-backed header and the hardware registers. A wrong address can corrupt stored calibration or hang the instrument.

Battery-Backed Memory

`DIAGnostic:CPF` (code $0BF) clears the power-up magic long at $E05000. The next power-up is then treated as a first power-up: the BSS area is zeroed, and the battery-backed header is reset. The serial number, options, system-version and processor-board-version strings are emptied, the backlight and contrast return to 70 and 33, and the GPIB address returns to 13. `DIAGnostic:BIST:BMONitor` (code $110) re-arms the RAM battery check word, the same routine as `SERVice:BIST:BMONitor` (Chapter 4).

Caution



`DIAG:CPF` erases the serial number, options and version strings at the next boot and resets the GPIB address. An empty processor-board version also selects the older self-test menu and disables the fan test (Chapter 4). Record these values first and write them back afterwards.

Firmware Download Entry

`DIAGnostic:DOWNload:FW:HPIB`, `:RS232` and `:RS422` (codes $0E7 to $0E9) hand the instrument to the boot ROM's downloader. Each one stops the timer, suspends the display and the other tasks, shows `Downloading over <port>...` on the display, stores a boot-monitor command line and a return address in low RAM, and transfers control. The instrument then waits for a firmware image on that port and does not return. The RS-232 and RS-422 forms return -241 Hardware missing (device code $80) if the serial interface is not fitted.

Caution



After one of these commands the instrument is waiting for a firmware image and responds to nothing else. Only a power cycle recovers it if no valid download follows. Do not send these commands unless a download is ready on the named port.

Debug Monitor and RTOS Reports

`DIAGnostic:MONitor` (code $0F9) executes a `TRAP #0`, which enters the boot ROM's debug monitor (Chapter 8); the console input routine runs the same trap on Ctrl-C. The pSOS reports `DIAGnostic:PSOS:PS`, `:SREP`, `:MREP`, `:MHREP` and `:SEMREP` (codes $103 to $107) print the kernel's task list, system report, memory-region report, heap report and semaphore report on the debug console. Interrupts are masked while each report prints, and nothing is returned over the remote interface.

Version Queries

`DIAG:VERS:FW?`	Code $11A. Returns the firmware build date and time, the compile time of version.c (`May 12 2009 10:17:44` on the E4419B).
`DIAG:VERS:BOOT?`	Code $11B. Returns the boot ROM version. The handler calls the boot ROM with `TRAP #14` and formats the result as `A.%05.2f`; the bench unit returns `A.02.00`.

Sensor EEPROM Access

E-series sensors carry an EEPROM that the measurement board reads over the instrument bus. These commands reach it directly. The data format is in Chapter 6.

Table 3-2. Sensor EEPROM commands

Command	Code	Behaviour
`DIAG:DET:EEPRom <addr>,<byte>`	$0DA	Writes one byte to the sensor EEPROM. No range or type check.
`DIAG:DET:EEPRom? <addr>,<n>`	$0DA	Reads 1 to 16 bytes as unseparated decimals; use n = 1.
`DIAG:DET:EEPRom:TEST` / `?`	$0C5	EEPROM write test over two address windows, restoring each. Query returns 1 or 0. E-series only.
`DIAG:DET:EEPRom:READ`	$0DF	Pauses measurement, re-reads the sensor data from the EEPROM, resumes.
`DIAG:DET:READEEPRom?`	$0E0	Re-parses the held EEPROM image and returns the parser status.
`DIAG:DET:IMAGe:HEADER?`	$0E1	Returns the model, serial, place, date and stinger revision from the sensor header.
`DIAG:DET:IMAGe:BLOCks?`	$0E2	Lists the installed data blocks, one `Block type n, format n` line each.
`DIAG:DET:EEPRom:RDI`	$0C4	Accepts and ignores a parameter.

Caution



`DIAG:DET:EEPR <addr>,<byte>` changes the sensor's own calibration store, with no range or type check. Read the whole EEPROM first (`DIAG:DET:EEPR? <addr>,1` for each byte) so it can be put back. `DIAG:DET:EEPR:TEST` also writes the EEPROM; it restores each window, so do not interrupt it or unplug the sensor while it runs.

A built-in dummy-EEPROM group (`DIAG:EEPROM:DUMMY:READALL?` and companions, codes $0C0 to $0C3) parses a fixed test image in RAM; `READDATA?` returns its argument, and `WRITEDATA` and `AUTOFILL` do nothing.

Sensor Calibration Image

A second group reads and writes the copy of the sensor's calibration tables held in RAM, not the EEPROM. Changes last until the sensor is re-read or disconnected. The tables use a pointer-and-entry scheme: a `POINter` command sets a running index and each `ENTRy` write stores at the index and advances it. For the V2P table, `DONe` recomputes the derived values.

- `DIAG:DET:IMAGe:V2P` (codes $0C6 to $0C8): the voltage-to-power linearity table, rows of 65 longwords.
- `DIAG:DET:IMAGe:TC` (codes $0CC, $0CD) and `:TCCoeffs` (codes $0D1, $0D3): the temperature-compensation word table and its coefficient table of doubles.
- `DIAG:DET:IMAGe:ZTC` queries (codes $0C9 to $0CB) and `:TC` measurement queries (codes $0CE to $0D0): zero- and temperature-compensation channel readings.
- Code $0D2 runs the sensor temperature-compensation update (`DIAG:TC:UPDate?`), and code $0DB reads and interpolates the cal-factor table (`DIAG:DET:IMAGe:CF`).

Caution



The `ENTRy` and `DONe` forms overwrite the working calibration tables. They are manufacturing and repair tools; a bad entry makes the channel read wrong until the sensor is re-read.

Measurement Board and DSP

These reach the measurement board and its DSP over the instrument bus (Chapter 5). The bus command words are in Table 5-1.

Table 3-3. Measurement-board and DSP commands

Command	Code	Behaviour
`DIAGnostic:DSP <addr>,<value>`	$0E5	Writes a 16-bit word into ADSP-2101 data memory (bus command $F804).
`DIAGnostic:DSP? <addr>`	$0E5	Reads a DSP data-memory word (bus command $F002).
`DIAGnostic:RADC?`	$102	Returns a raw 4-byte ADC reading.
`DIAG:DET:TEMPerature?`	$0D8	Returns the sensor thermistor reading.
`DIAG:DET:MRESistor?`	$0D9	Returns the sensor mount-resistor measurement (sensor-type ID).
`DIAG:DET:GSWitch <1|2>`	$0D5	Forces detector gain path 1 or 2. Always returns -310, apparently to flag the override.
`DIAG:DET:SPEed <bool>`	$0D6	Sets the board speed bit.
`DIAG:DET:RCALibrate <bool>`	$0D7	Sets the reference-calibrator bit.
`DIAGnostic:RCALibrate <bool>`	$0F7	Switches the RCAL path on the board.
`DIAG:DET:THIRd`	$0D4	E-series with the flag set: writes DSP $3815 = $105, runs one cycle, writes $3815 = 0.
`DIAGnostic:SRATe <bool>/?`	$0FA	Selects one of two sample-period constants and recomputes the filter length.

Measurement Parameter Overrides

`DIAGnostic:SETVar <name>,<value>` and `DIAGnostic:SETVar? <name>` (code $0E6) read and write named measurement variables, some in the channel state and some in the sensor structure.

Table 3-4. DIAGnostic:SETVar names

Name	Touches	Name	Touches
MEAS	measurement value	VTHerm	thermistor voltage
G_SENSOR	sensor gain	RCAL_DELAY	reference-cal delay
CAL_OFFSET	cal offset	FC_VALUE	freq-cal value (query)
RAW_ADC	raw ADC	TC_THRESHOLD	temp-comp threshold
SENSOR_RANGE	gain path 1 or 2	POSTAVE_MEAS	post-average (query)
TC_VALUE	temp-comp (query)	PREAVE_MEAS	pre-average (query)
ZERO_LOWER / UPPER	zero offsets	CAL_CHAR	cal characteristic

Other per-channel overrides set the filter length used during zeroing, reference-cal and calibration (`DIAG:AVERage` codes $0F1 to $0F6), a reference-pad attenuation (`DIAG:RPAD`), and various sensor-detector and zero-drift test limits. Most store into the sensor or channel structure and are read back by the matching query.

`DIAG:CALibration:CHECk`, `:ZERO:CHECk`, `:ZERO:DRIFt:CHECk` and `:STIMe:ANALog:FOURty` store their parameter in an internal variable that the matching query reads back. `:STIMe:ANALog:TWENty`, `:STIMe:DSP`, `DIAG:TEST`, `DIAG:BIST:POST` and `DIAG:RCALibrate:SFACtor:DTYPe` (code $0F8) accept a parameter and do nothing.

Status Reports

`DIAGnostic:SETTings?` (code $0E3) returns a text block of the channel's measurement settings: cal factor, reference cal factor, measurement and configured frequency, units, continuous-init and trigger state, and window resolution. `DIAGnostic:STATes?` (code $0E4) returns the channel's mode, waiting flag, trigger state and next state, and the sensor and its position. Both return human-readable text.

Display

Table 3-5. Display commands

Command	Code	Behaviour
`DIAG:DISPlay:DUMP? <row>`	$0FB	Returns 40 bytes of SED1330 memory from row×40, one 320-pixel line.
`DIAG:DISPlay:BRIGhtness:UP` / `:DOWN`	$0FC/$0FD	Step the backlight drive.
`DIAG:DISPlay:BRIGhtness:SET?`	$0FE	Calibrates the backlight current, saves it in battery-backed RAM, returns it. Takes several seconds.
`DIAG:DISPlay:CONTrast:UP` / `:DOWN`	$0FF/$100	Step the LCD contrast.
`DIAG:DISPlay:CONTrast:SET`	$101	Saves the contrast step to battery-backed RAM.
`DIAG:DISPlay:DBUFF <bool>` / `?`	$108	Double-buffer flag.
`DIAG:DISPlay:BUFF <1|2>`	$109	Selects display page 1 or 2.
`DIAG:DISPlay:RBUFferring`	$10A	Selects display page 2.
`DIAG:BIST:DISPlay:TEST <n>`	$115	Runs display test pattern n.

A full screen is rows 0 to 207 of `DIAG:DISP:DUMP?`, 40 bytes per row with the bits least significant first across the line. Chapter 9 gives the menu paths.

Keyboard, Battery and Self Tests

Most of these commands run the same workers as the front-panel Individual self tests (Chapter 9) and return a result byte or text.

- `DIAG:BIST:KEYBoard:DISable` / `:ENABle` (codes $116, $117) lock and unlock the front-panel keys; `:KEY?` (code $119) returns the last key code.
- `DIAG:BATTery:FITted?`, `:ABSent?`, `:CAPacity?`, `:CHARge?`, `:TIME?` (codes $10B to $10F) report the optional battery pack.
- `DIAG:BIST:MEAS?` (code $111) runs the measurement-board self test; `DIAG:BIST:ANALog? <point>` (code $112) reads one analog test point (THERMA, LED_I_MON, REF_TP1/2 and others).
- `DIAG:RECorder:TEST?`, `DIAG:TTL:TEST?`, `DIAG:RS232:TEST?` and `DIAG:RS422:TEST?` (codes $113, $114, $122, $123) run the recorder, TTL and serial loop-back tests. The serial tests run only while internal variable $AE is 0 (otherwise -221), and the RS-422 test needs the RS-422 interface fitted.
- `DIAG:UART:REGS?` (code $124) returns the 16550 register dump. `DIAG:RMTIO:TEST?` (code $127) always returns a pass; the test is not implemented.

Interface and Hardware Selection

- `DIAGnostic:PLD:MODE` (code $128) writes the PLD mode register, 0 to $F01180 or 1 to $F011C0, from internal variable $1D.
- `DIAGnostic:MONGO:PORT <string>` (code $129) passes a port-description string to the sensor-port manager; the default is `Sensors Connected Front and Rear`.
- `DIAGnostic:HADDRess` is an alias of the GPIB-address command.
- `DIAGnostic:RECorder:CHANnel` (code $029) chooses which channel drives the recorder output; it reports `HARDWARE MISSING` for channel B on a single-channel unit.
- `DIAGnostic:UNIT:SCPIPOWER` and `:SCPIRATIO` (codes $0BC, $0BD): the set forms do nothing; the queries return internal unit variables or their MIN/MAX companions. `:OLDWATTS` accepts a parameter and does nothing.

Note



Command codes $01B to $039 (apart from the recorder commands $029 to $02D), and a few others, have handlers but no keyword reaches them in this build, so they cannot be sent. They include an FDO-table pair and alternate measurement routines for A- and D-type sensors.

4

Other Undocumented Commands

SERVice Subsystem

These sit alongside the documented SERVice commands.

SERV:BIST:BMONitor	Code $00A. Writes $12345678 to the RAM battery check word at $E05070. The RAM Battery self test reports Passed only while that word holds the magic value, so this command clears a RAM Battery Fault after the backup battery has been replaced.
SERV:BIST:CALibrator	Code $014. Accepts its parameter and does nothing.
SERV:SENSor:TNUMber?	Code $011. Returns the 16-character test-number string from an E-series sensor EEPROM. Confirmed on the bench (returns the sensor serial).
SERV:SENS:FREQ:MAX? and companions	Codes $015 to $01A. Return frequency and power limits from the extended E-series EEPROM block. A sensor without that block (such as the N8482A tested) returns -241 with Sensor connected does not contain this information.

STATus:DEBug Subsystem

STAT:DEB:HAP <n>	Code $077, long form STATus:DEBug:HAPpenning. Returns its argument as the handler status, so the parser reports internal code n as if a real command had failed with it. It exercises the error queue and status reporting; Chapter 10 lists the codes.
STAT:DEB:ENBALL	Code $078. Sets the enable mask of every status register set to $7FFF, so every event propagates to the status byte.

SENSe:CORRection:STATe

Code $146. The Programming Guide documents several :STATe commands under this node but not this one. The set form stores a per-channel boolean in internal variable $2A and returns -114 for a channel the model does not have; the query returns the boolean. No other code reads that variable, so the setting appears to change nothing. On the E4419B both SENS:CORR:STAT? and SENS2:CORR:STAT? return 0.

SENSe:CORRection:LOSS1

Code $14D. The Programming Guide documents `CFACtor|GAIN1` (code $149) but mentions LOSS1 only in a note, as being for calibration factors. LOSS1 is the cal factor expressed as a loss, LOSS1 = 100 – CF in percent, with a range of -50 to 99; CFACtor and GAIN1 have a range of 1 to 150. Internally the limits are held as fractions: 0.01 and 1.5 for the cal factor, -0.5 and 0.99 for the loss (Chapter 7).

LOSS1 and CFACtor work only with a sensor that needs a manual cal factor. For an E-series sensor the handlers return -241 Hardware missing, unless its stinger revision bytes are '4' and '1', '3' or '5' (shown by `DIAG:DET:IMAG:HEADER?` as 52.49, 52.51 or 52.53; the N8482A reads 52.48). Confirmed on the E4419B with an 8481A on channel B: `CFAC?` 100 (MIN 1, MAX 150), `LOSS1?` 0 (MIN -50, MAX 99); `LOSS1 2` set `CFAC?` and `GAIN1?` to 98, and `LOSS1 -20` set `CFAC?` to 120. With the N8482A the same queries return -241. Not described in the Programming Guide.

MEMory:TABLe:LOSS

Code $170. This is the loss counterpart of the documented `MEMory:TABLe:GAIN[:MAGNitude]`, meant to enter a sensor-calibration or frequency-offset table as loss rather than gain, the table form of LOSS1 above. It does not work, for three reasons, each confirmed on the E4419B:

- **No percent conversion.** GAIN values arrive as fractions (98% as 0.98); LOSS values are checked against the LOSS1 limits 0.99 and -0.5, so `MEM:TABL:LOSS 2` fails with -222 and a 2% loss must be sent as 0.02.
- **Wrong stored value.** Each value v is stored as 100 minus v where 1 minus v was intended, and the table store keeps round(1000×value) in 16 bits, so a near-100 value wraps.
- **Wrong readback.** The query reports 100 plus each value that `GAIN?` would report, where 100 minus it was intended.

On the bench, the built-in HP8482A table reads back as `GAIN?` 98.0, 98.0, 99.5 and as `LOSS?` 198.0, 198.0, 199.5. On an empty table, `MEM:TABL:LOSS 0.02,0.5` was accepted, after which `GAIN?` returned 3444.4 and 3396.4 and `LOSS?` 3544.4 and 3496.4, as the arithmetic predicts. Use `GAIN` with cal factors in percent; do not use `LOSS`.

Note



Once a table has been selected with `MEMory:TABLe:SELect` there is no SCPI way back to no-table-selected: `SELect ""` is rejected with -224. The selection only sets the target for table edits; it does not apply a correction to measurements.

Stub Commands

Three commands do nothing useful. `TCONF <parameter>` (code $0B1) returns success. `FORMat:SREGister` (code $0BB) reads the `FORMat:DATA` setting and returns success. `RTEST<n>:ONE?` and its siblings (code $12F) share the `FETCh?` handler and only set the channel number it sees, including non-existent channels 4 and 5; they look like a channel-range test hook. Use `FETCh?` instead.

Version-String Side Effects

Two documented `SERVice:VERSion` writes have undocumented side effects, because the firmware reads the stored strings as feature switches.

`SERV:VERS:PROCessor`	The processor-board version string is looked up in a table at $E05144: below revision C it reads 0, at C or later 1. When 1, the Individual self-test menu shows the Memory / Lithium Battery / Fan set and the fan test runs; when 0, it shows the ROM Checksum / RAM set and the fan test does nothing. The bench E4419B reports revision C.
`SERV:VERS:SYSTem`	The system version string is compared with `2`: a match selects PLD mode 1 (write to $F011C0), anything else mode 0 (write to $F01180). This looks like a processor-board variant switch.

Caution



Both strings change instrument behaviour beyond the Version screen. If you write either one, record the original first and set it back.

5

The Measurement Board and DSP

Bus Protocol

Every operation on a measurement board goes through one transfer routine that takes a board, a 16-bit command word, and transmit and receive buffers. Bits 15 to 11 of the command word are the opcode, bit 10 the channel, and the low bits a length or sub-function. The command words seen from the callers:

Table 5-1. Measurement-board bus commands

Command word	Purpose
$0800	Board identify / BIST status (power-up scan)
(ch<<10)|$0801	Read channel status word
(ch<<10)|$1000	Channel command, no data
(ch<<10)|$1800	Read 4-byte raw measurement (zero, cal, RADC?)
(ch<<10)|$2002	Write 2-byte channel setting
(ch<<10)|$2800|cfg	Channel configuration bits (Table 5-2)
(ch<<10)|$3004	Write one sensor EEPROM byte
(ch<<10)|$3800|n	Read n sensor EEPROM bytes
$D800	10-byte read: DSP and gate-array revision strings
$E000|n	Analog measurement: mount resistor
$E001	Analog multiplexer reading
$E005	Sensor temperature
$F002	Read ADSP-2101 data-memory word
$F804	Write ADSP-2101 data-memory word

Channel Configuration Bits

The configuration command (opcode $2800) carries a config byte in its low 8 bits. Each of four wrappers sets or clears one bit and resends the whole byte, so the bits accumulate. Four undocumented commands drive the four bits:

Table 5-2. Configuration byte bits (opcode $2800)

Bit	Mask	SCPI command	Meaning
0	$01	`DIAG:DET:SPEed`	Measurement speed select
1	$02	`DIAG:RCALibrate`	RCAL signal path
2	$04	`DIAG:DET:RCALibrate`	Reference calibrator on
3	$08	`DIAG:DET:GSWitch`	Detector gain path

The ADSP-2101 Program

The measurement board's DSP firmware is a separate 64 KB image (part E4418-80023). It is organised as eight 2048-word boot pages, matching the ADSP-2101's boot structure. Each 24-bit instruction word is stored big-endian in three bytes with a fourth byte that is $FF over code.

Table 5-3. DSP image page map

Page	Words	Content
0	0 to 2047	Boot page: an 11-word loader plus a 781-word data block it copies into data memory
1	2048 to 4095	Main code (~2021 words), loaded over page 0 as the running overlay
2 to 6	4096 to 14335	Empty
7	14336 to 16383	Top 3,856 bytes are the XC3042 gate-array bitstream; no DSP code

At reset the ADSP-2101 loads page 0 and runs the loader, which copies the 781-word data block down into data memory at $3802 to $3B0E and then writes the System Control Register to force the boot load of page 1. Page 1 opens with the interrupt vector table and is the sensor-acquisition loop: it polls the board hardware, runs ALU and MAC filtering, and exchanges commands and results with the main processor through a data-memory mailbox.

The code page is fully disassembled (every word decodes to a known ADSP-2100-family instruction). Seven words read an undefined group-3 register and are followed by a software-stack push, with an exit of `I6 = MR1; JUMP (I6)`. The prologues and epilogues indicate that these words pop the return address off the hardware PC stack, the operation later parts name TOPPCSTACK. Three of the seven prologues never execute: the interrupt vectors enter those routines below the prologue, and they exit by RTI or RTS.

The Command Mailbox

The main processor writes DSP data-memory words with bus command $F804 and reads them with $F002; `DIAGnostic:DSP` exposes both. The mailbox locations:

Table 5-4. DSP data-memory mailbox

DM address	Use
$0002, $0006, $0007	Board hardware status, polled in the loop
$0800, $0803	Board hardware control registers
$3800 onward	Circular sample buffer
$3815	Command from the main CPU (DIAG:DET:THIRd writes $105 here)
$381E, $381F	Status the DSP writes back; the main CPU reads with $F002
$3844 to $388F	Working and result area

On the bench, `DIAG:DSP?` reads of $381E, $381F and $3815 return 9, 0 and 0. The roles of these words come from the DSP disassembly, which reads $3815 and writes $381E/$381F.

The DSP Dispatch Table

The DSP's serial-port receive interrupt stores each incoming word, and the main loop splits it into the opcode (bits 15 to 11) and the low bits, then jumps through a 31-entry table indexed by opcode minus one. This is the DSP side of the bus protocol in Table 5-1.

Table 5-5. DSP opcode handlers

Opcode	Command	Action
1	$0800	Board identify / BIST status
2 to 4	$1000 to $2000	Channel command, raw read, channel setting
5	$2800	Configuration: low bits index a second 20-entry table
6, 7	$3000, $3800	Sensor EEPROM write and read
8 to 26	$4000 to $D000	No action
27	$D800	Sends the 10 revision bytes `a0111a0103`
28, 29	$E000, $E800	Analog measurement, transfer control
30, 31	$F000, $F800	Read and write a data-memory word

Opcode 27 is the source of the Version screen's per-channel DSP Revision and Gate Array Rev lines: the main processor reads the 10 bytes and formats `a0111` as A.01.11 and `a0103` as A.01.03. Both come from the DSP's own data, not from the gate array.

The XC3042 Gate Array

The top 3,856 bytes of the DSP flash (not DSP code) are the configuration bitstream for the measurement board's Xilinx XC3042A gate array. Read from the top address downward, least significant bit first, they form a complete XC3000-family stream: a preamble, a 24-bit length count of 30,825, then 285 frames of 108 bits (the XC3042 layout), then a postamble.

The schematic (CLIP page 66) confirms the mode. The gate array's mode pins are M0 pulled low and M1 and M2 pulled high, which is XC3000 master-parallel-down: the part drives its own address lines from the top of the shared flash downward and loads itself at power-up, with no help from either processor. Its address and data lines sit on the same bus as the DSP flash. An unfitted header brings out the serial-configuration pins, so a download cable could override the mode and load the part serially.

6

Sensor EEPROM Data Format

Container Layout

An E-series ("Stinger") sensor carries an EEPROM that the measurement board reads over the bus, 16 bytes at a time. The main firmware parses it into a RAM sensor image. The container:

Table 6-1. Sensor EEPROM layout

Address	Content
0	Format/revision byte. The parser accepts 1, 2, 3, 6 or 7 and retries up to five times otherwise; 0, 4, 5 and 8 or above are unreadable.
1	Sub-revision byte
2 to 3	One word
4	Block count N
5 onward	N directory entries, 7 bytes each: {u16 blockAddr, u8 type, u8 format, u16 length, u8 checksum}

Each block's payload lives at its `blockAddr`, `length` bytes, protected by an 8-bit additive checksum: the sum of the payload bytes must equal the entry's checksum byte, or the parser prints `EEPROM - data block failed checksum`.

Block-Type Registry

The parser registers a handler per block type and format, each with a name. The complete set:

Table 6-2. Sensor EEPROM block types

Type	Name	Formats	Severity
1	Linearity	1	Required
2	Temp-Comp	1, 2	Required
3	Cal Factor	1, 2	Required
4	Header	1	Required
5	Misc	1, 2, 3	Required
6	Basic Params	1, 2	Optional
8	Auto-Filter	1	Optional

Types 7 and 9 to 20 are not used. The Linearity, Temp-Comp and Cal Factor blocks are the calibration tables reached by the `DIAG:DET:IMAGe` commands in Chapter 3. The header block holds the model, serial, place, date and stinger revision returned by `DIAG:DET:IMAGe:HEADER?`.

Severity and Read Failures

A severity byte in each type's registration controls what a missing or bad block does:

- **Required (types 1 to 5):** a missing or bad block queues `Sensor EEPROM <name> data not found or unreadable` and the read ends with `Read Failed - critical`.
- **Optional (types 6 and 8):** if the directory does not list the type, the read continues with no message. Only a listed block that fails its checksum is treated as critical.

The N8482A tested carries no type 6 or 8 block and reads cleanly. Its `DIAG:DET:IMAG:BLOC?` lists type/format 1/1, 2/2, 3/1, 4/1 and 5/2; see Chapter 11.

The N8482A EEPROM

The EEPROM of the N8482A tested (serial MY51520008) was read out byte by byte with `DIAG:DET:EEPR? <addr>,1`. It is format 6 with seven directory entries, and every block matches its directory checksum:

Table 6-3. N8482A EEPROM directory

Type	Name	Format	Address	Length
1	Linearity	1	54	195
2	Temp-Comp	2	249	286
3	Cal Factor	1	535	185
4	Header	1	2144	92
5	Misc	2	2236	31
18	(not registered)	1	2267	5
19	(not registered)	1	2272	5

Types 18 and 19 hold five bytes of $AA each. The firmware registers no handler for them, so the parser skips them and `IMAG:BLOC?` lists only types 1 to 5. Everything from byte 2277 to at least 8191 reads $FF.

The Cal Factor block is a 9-byte header, with the point count at byte 5, followed by 8-byte records: a 32-bit frequency in kHz and two 16-bit factors in which $4000 is 100%. In this sensor both factors are equal. The 22 points run from 100 kHz (97.28%) through 50 MHz (100.00%) to 6 GHz (94.67%), and `DIAG:DET:IMAG:CF:NPTS?` returns 22. The 32 bytes after the block hold four more records in the same format, 15 to 18 GHz at 100%, outside the point count and the sensor's range; the meter does not read them.

7

Firmware and Memory

The Battery-Backed Header

The locations below survive power-off and firmware downloads. Their first-power-up defaults are set during boot.

Table 7-1. Battery-backed header ($E05000)

Address	Content	Command
$E05000	Power-up magic $12345678	`DIAG:CPF` clears it
$E05004	System version string	`SERV:VERS:SYST`
$E05018	Processor-board version string	`SERV:VERS:PROC`
$E0502C	Serial number string	`SERV:SNUM`, Edit SN
$E0504A	Options string	`SERV:OPT`
$E05068	Backlight drive (default 70)	`DIAG:DISP:BRIG:SET?`
$E0506C	Contrast (default 33)	`DIAG:DISP:CONT:SET`
$E05070	RAM battery check word	`SERV:BIST:BMON`
$E05074	GPIB address (default 13)	`SYST:COMM:GPIB:ADDR`

The Internal Variable Store

Commands and the front panel keep settings in numbered variables (0 to $11D) held by an "istate" object, built from a descriptor block in the image. Each descriptor gives a signed offset, a size and a type, so a variable's default can be read straight from the ROM image. The defaults explain several behaviours in this supplement: variables $CC and $CD are the cal-factor limits 1.5 and 0.01 (150% and 1% held as fractions), and $D5 and $D6 are the LOSS1 limits 0.99 and -0.5 that the broken `MEMory:TABLe:LOSS` checks against (Chapter 4).

SCPI Parser Structures

The parser is table-driven. A keyword entry (42 bytes) holds a pointer to the keyword text, a child group, a channel flag and a pointer to the 16-bit command code. A group holds an id, a count, a default index and the child entry pointers. The command code indexes a descriptor table whose entries point to a set record and a query record; a record either names the handler directly, points to a parameter-descriptor block, or marks a query that returns a bound internal variable. This is the machinery behind the command-code model in Chapter 2.

Command-Set Coverage

The E4418B tree reaches 314 command codes plus 16 common commands; the E4419B reaches 330, the extra ones being the dual-channel DIFFerence and RATio forms. About 150 codes appear in none of the manuals, most of them in the `DIAGnostic` subtree. A keyword-by-keyword comparison against the Programming Guide places every reachable code either in the manuals or in this supplement. Two undocumented commands sit inside documented subsystems: `[SENSe]:CORRection:STATe` and `MEMory:TABLe:LOSS` (Chapter 4).

8

The Boot ROM

Identification

The boot ROM occupies the bottom 128 KB of the two firmware flash chips, $000000 to $01FFFF. Two read-outs of the bench E4419B over GPIB with `DIAGnostic:PEEK:BLOCK?` are identical. Code and data run from $0000 to $A1AF; the rest is erased. The ROM identifies itself as `Bootrom Revision A.02.00`, the same value `DIAG:VERS:BOOT?` returns, and carries the ID string `@(#)LDS Rev: 2.03 ; Bootrom Rev 1.02` and a Hewlett-Packard copyright dated 1988 to 1996. Its flash programmer recognises three device IDs: SGS-Thomson 29F040, Fujitsu 29F040A and AMD 29F040.

Power-Up Modes

The boot ROM owns the exception vectors (it sets VBR to $00000A) and decides what runs at power-up:

1. It scans four front-panel key rows. A key held on a given row selects a special mode: HPIB download, RS-232 download, RS-422 download, or clearing memory (which wipes RAM, including the battery-backed area).
2. It polls the console up to 1,000 times. ESC or Ctrl-F in that window enters the ROM monitor.
3. Otherwise it prints `Starting main firmware...` and jumps to $020000.

Caution



One of the power-up key rows clears all RAM, including the battery-backed settings and user calibration tables. Which physical keys sit on these rows has not been identified. Do not hold front-panel keys at power-up while investigating, and do not experiment with the memory-clear mode on an instrument whose calibration you want to keep.

The console here is the MC68332's on-chip SCI, not the 16550 UART the main firmware uses for its serial option. Where the SCI pins come out, and the baud rate, have not been traced.

The ROM Monitor

The monitor prompt is `ROM Monitor / Enter ? for help.` The main firmware enters it with `DIAGnostic:MONitor` and on Ctrl-C from its console. Its commands:

Table 8-1. ROM monitor commands

Command	Purpose
bootvars	Display boot ROM variables
bc [<hex>]	Set the boot configuration (disabled in this build)
gu [<hex addr>]	Go to start address
bs, gbreak, gdb	Breakpoint and gdb-trap controls
dlong / dword / dbyte / dmem	Display memory
slong / sword / sbyte / smem	Set memory
hmon [device]	Download into memory
version	Display boot ROM version

A gdb remote stub is included (read and write memory packets, checksum handling). The boot-config store is disabled in this build: its area size is zero, so any `bc` write ends in `bootConfigAreaSize is zero`.

Download Record Format

The loader reads two-character record tags from the selected port. The power-up download paths and the firmware's `DIAGnostic:DOWNload:FW` commands both drive it through the monitor's `hmon` command.

Table 8-2. Download record tags

Tag	Payload	Action
WL	u32 start, u32 end, then the data	Write and load; flash addresses go through the programmer, others to RAM
GU	u32 address	Run address, kept with the download state
H0	none	End of download

This is the structure of the `.dload` files in Chapter 1: a `WL` record for the code, a `WL` record for the data, and the `H0` terminator, which is the `48 30` trailer.

9

Front-Panel Service Functions

The Softkey Menu Engine

The current screen is held as a menu id in a state variable. A softkey builder switches on that id and installs a softkey descriptor set through a menu-manager object; a softkey press runs the descriptor's action or pushes a child screen id. Softkey labels are stored as ids into a flat label table, not as strings. The leaf actions come from the code; the navigation paths below were mapped on the E4419B by reading the LCD over GPIB while stepping the panel.

Reaching the Service Items

The four softkeys are to the right of the display (SK1 top to SK4 bottom); the hard keys are along the bottom edge. The service items are reached as follows on the E4419B. Of these, only Edit Serial Number is not in the User's Guide.

Table 9-1. Front-panel paths to the service items

Item	Path
Edit Serial Number	System/Inputs > More > Service > Version > Edit Serial Number
Individual self tests	System/Inputs > More > Service > Self Test > Individual
Command Set / language	System/Inputs > Remote Interface > Command Set
Power reference on/off	System/Inputs > More > Power Ref

The Service menu has Self Test, Display, Version and Serial Diagnostic (greyed out without the serial option). The Version screen shows the model, serial, firmware and boot-ROM revisions, the processor-board and system version strings, and the per-channel DSP and gate-array revisions; its softkeys are Edit Serial Number and Done.

Edit Serial Number

The Edit Serial Number softkey copies the entry buffer into the serial-number variable at $E0502C, the same store `SERVice:SNUMber` writes. There is no format check: the "Invalid Serial Number" branch is unreachable because the check that guards it always passes.

Caution



Edit Serial Number overwrites the stored serial number with no validation and no way to undo it. Write down the original before using it.

The Individual Self Tests

The firmware holds two Individual self-test softkey sets and chooses between them by the processor-board version string (Chapter 4). A board at revision C or later, such as the bench E4419B, shows the first set below.

Table 9-2. Individual self tests

Set (revision C+)	Set (earlier boards)
Memory	ROM Checksum
Lithium Battery	RAM
Measure Assembly	RAM Battery
Calibrator	Measure Assembly
Keyboard	Calibrator
Fan	Keyboard
Display	(serial items)
Serial Interface	

Most softkeys run the same worker as a `DIAGnostic:BIST` query; the Fan test and the front-panel loop-back tests have no remote counterpart. The Memory test runs the ROM check and then the RAM check. The serial loop-back tests need a rear-panel loop-back connector, and the RS-422 test also needs the RS-422 option fitted.

The HP436A Modes Menu

While `SYSTem:LANGuage HP436A` is selected on the E4418B, the front panel offers an **HP436A Modes** softkey. It opens a menu with two softkeys, **Zero Mode** and **Filter Mode**, which set the emulation state that the 436A reading formatter uses (Chapter 2). From firmware analysis; not exercised.

The Test1 to Test6 Screen

A separate developer screen holds Test1 to Test6. These are not instrument self tests: each Test softkey calls a display-field format routine that writes a widest-case sample string such as `CF:  199.9%` or `999.9GHz(M)` into the frequency and cal-factor display fields, to check the field layout. The screen does not appear in normal navigation and looks like a dormant engineering aid. Two unlabelled items in the same menu block ($05DCC2 and $05DD1C) search the options string for `001` and open one of two menus depending on the result.

Note



This screen is easy to mistake for the Individual self-test menu because of the `Test` labels. It runs no test and reports no result; it only exercises the display fields.

10

Device Error Codes

The Two Translation Tables

A command handler returns a small internal code, which the dispatcher translates to a SCPI error through one of two tables by sign. Negative internal codes use a static table at $0C656C (110 entries, codes 0 to -109). Positive device-specific codes use a table in initialised data, at $E0EC38 on the E4418B and $E0B8EC on the E4419B. Both tables have the same 6-byte entry layout {u16 type, s16 scpi_error, s16 status-bit}: a type of 3 queues the SCPI error, and types 0 to 2 set, clear or toggle a status bit.

The negative table was confirmed live with `STAT:DEB:HAP`: internal -11 queues -222, -57 queues -224, -8 queues -200.

Positive Device Codes

The positive device codes and their SCPI translations. Codes $7A to $82 and $86 were checked on the E4419B; the others are from the table in the image.

Table 10-1. Positive device error codes

Code	SCPI	Meaning	Code	SCPI	Meaning
$7A	-211	Trigger ignored	$82	-256	File name not found
$7B	-213	Init ignored	$83	-257	File name error
$7C	-214	Trigger deadlock	$84	-226	Lists not same length
$7D	-114	Header suffix out of range	$85	-330	Self-test failed
$7E	-221	Settings conflict	$86	-231	Data questionable
$7F	-230	Data corrupt or stale	$89	-361	Parity error
$80	-241	Hardware missing	$8A	-362	Framing error
$81	-220	Parameter error	$8B	-363	Input buffer overrun

Codes $87 and $88 change a status bit and queue nothing. The code backs these up: handlers return $7D for an out-of-range channel suffix, $80 when the sensor is not an E-series part, and $7E for a `MEMory:TABLe` command with no table selected.

11

Verification Record

Confirmed by Observation

The findings below were checked on a real E4419B (serial MY45103686, firmware A2.09.01, boot ROM A.02.00) over GPIB, with an N8482A sensor on Channel A and the power reference, and separately an 8481A on Channel B. Writes were limited to settings restored afterwards (a scratch user table and the 8481A cal factor), the table-edit selection, and normal zeroing and calibration. Nothing was sent that writes the sensor EEPROM, the battery-backed header or the firmware, enters download mode or switches language.

- **Identity and versions.** `*IDN?`, `DIAG:VERS:FW?` and `DIAG:VERS:BOOT?` match the analysed image (build 12 May 2009, boot ROM A.02.00). The Version screen read off the LCD confirms the processor-board version C, system version 2, and per-channel DSP A.01.11 and gate-array A.01.03.
- **Memory access.** `PEEK` and `PEEK:BLOCK?` returned the GPIB address, the power-up magic and the error-table entries, matching the two-record download image, and the 128 KB boot ROM (two identical read-outs).
- **Installed firmware.** The main firmware flash, $020000 to $0FFFFF, read with `PEEK:BLOCK?`, is identical byte for byte to the A2.09.01 download file loaded as two records ($020000 to $0F1D5B). The remaining 58,020 bytes up to $0FFFFF are erased ($FF). With the boot ROM, the whole 1 MB of flash is accounted for.
- **Error tables.** `STAT:DEB:HAP` queued the expected SCPI error for every negative code tested and for positive codes $7A to $82 and $86.
- **Sensor EEPROM.** `DIAG:DET:IMAG:HEADER?` and `:BLOC?` returned the N8482A identity and block list (type/format 1/1, 2/2, 3/1, 4/1, 5/2), matching the format in Chapter 6. The undocumented `SERV:SENS:TNUM?` returned MY51520008, the same string as the sensor serial number. Two byte-by-byte read-outs of the raw EEPROM with `DIAG:DET:EEPR?` were identical, and every block matched its directory checksum (Chapter 6).
- **DSP mailbox.** `DIAG:DSP?` reads of $381E, $381F and $3815 returned 9, 0 and 0 (Chapter 5).
- **Calibration path.** Zeroing and calibration completed with no error on the N8482A and on the 8481A. With the 8481A on the 50 MHz power reference and `OUTP:ROSC ON`, five readings fell between +0.00002 and +0.0021 dBm; with the reference off the reading fell to the noise floor, about -39 to -41 dBm.
- **Cal factor and LOSS1.** With the 8481A, `CFAC?` limits were 1 and 150, `LOSS1?` limits -50 and 99, and `LOSS1 2` and `LOSS1 -20` gave cal factors of 98 and 120. The original value was restored.
- **Other commands.** `SENS:CORR:STAT?` and `SENS2:CORR:STAT?` return 0. `MEM:TABL:SEL ""` is rejected with -224. `MEM:TABL:LOSS?` with no table selected returns -221.

- **MEMory:TABLe:LOSS.** After `MEM:TABL:LOSS 0.02,0.5` on an empty user table, `GAIN?` returned 3444.4 and 3396.4 and `LOSS?` 3544.4 and 3496.4, showing both the store and the readback faults. The table was cleared afterwards.
- **Aliases and dormant entries.** `READD?` behaves as `READ?` on the E4419B; `*_TRG?` returns -101.

Not Verified

The following rest on firmware analysis alone and were not exercised:

- Everything that writes non-volatile state: `POKE`, `DET:EEPRom` writes, `CPF`, `DOWNload:FW`, the `SERVice:VERSion` and `SNUMber` writes, and the display and variable writes.
- The HP 436A emulation code path (the decoder is read from the image, not exercised).
- The boot ROM power-up key rows: which physical key selects each download or memory-clear mode, the SCI console pins and its baud rate.
- The meaning of the DSP's 781-word data block beyond its revision strings and dispatch tables, and the DSP configuration sub-table entries.
- The `SERVice:SENSor` frequency and power-limit queries with a sensor that carries the extended EEPROM block (the N8482A does not).

How to Reproduce

The bench checks were run from Python over GPIB (pyVISA) at address 13. Reproducing them needs the instrument on the bus, a sensor for the sensor and calibration items, and the memory-access commands in Chapter 3. The error tables are the quickest confirmation: send `STAT:DEB:HAP <n>` followed by `SYST:ERR?` and compare with Table 10-1. For the front-panel map, `DIAG:DISP:DUMP?` over rows 0 to 207 renders the screen (40 bytes per row, bits least-significant-first), and a go-to-local between captures keeps the panel usable. Addresses on another firmware revision will differ; re-derive them from that image rather than trusting the values here.